Transparent Decorator Encryption with Dual-Read Migration for Multi-Tenant Storage

Tom Tang
Edge Vector Foundation

2026

Abstract

A system and method for providing transparent, authenticated encryption of data at rest in a multi-backend, multi-tenant storage system. The invention introduces an encrypting decorator layer that wraps any key-value storage backend—including embedded databases (Sled), cloud-hosted tables (DynamoDB), and in-memory stores—without modifying the underlying backend implementation. The decorator intercepts all read and write operations: on write, values are encrypted using AES-256-GCM authenticated encryption and stored in a self-describing format `ENC:<base64(nonce||ciphertext||tag)>` that remains valid UTF-8 for compatibility with string-typed storage backends; on read, the decorator detects the `ENC:` prefix and decrypts transparently. A dual-read migration mode enables zero-downtime transition from plaintext to encrypted storage: when decryption fails or the `ENC:` prefix is absent, the system reads the value through unchanged as plaintext, while all new writes are always encrypted. This eliminates the need for batch migration or downtime during encryption rollout. Per-tenant cryptographic isolation is achieved through a multi-layer key hierarchy: an AWS KMS Customer Master Key (CMK) protects a Key Encryption Key (KEK), which is used via HKDF to derive per-tenant Data Encryption Keys (DEKs) using the tenant's SHA-256 hash as the derivation context. Namespace-aware encryption ensures that each logical namespace (schemas, metadata, main data, transforms) uses independent key derivation paths. The decorator pattern is composable: multiple decorators (encrypting, caching, logging) can be stacked without backend awareness, and key rotation is supported by encrypting new DEKs under a rotated KEK while retaining old DEKs for reads until all data is re-encrypted.

1 Introduction

The present invention relates generally to data encryption systems for multi-tenant storage platforms, and more particularly to methods and systems for implementing transparent encryption as a composable decorator layer over heterogeneous key-value storage backends, enabling zero-downtime migration from plaintext to encrypted storage through a dual-read mechanism, with per-tenant key isolation derived from a hierarchical key management architecture.

2 Background

2.1 The Encryption-at-Rest Problem in Multi-Backend Systems

Modern data platforms increasingly support multiple storage backends to serve different deployment scenarios: embedded databases for local single-user operation, cloud-hosted key-value stores for multi-tenant production, and in-memory stores for testing and development. Adding encryption

at rest to such systems presents a fundamental architectural challenge: the encryption logic must work identically across all backends without coupling to any specific backend's implementation.

Existing approaches to storage encryption suffer from several deficiencies:

Backend-specific encryption. Many cloud storage services (e.g., DynamoDB server-side encryption, S3 SSE) provide encryption at rest as a backend-specific feature. This approach ties the encryption to a single backend, provides no encryption for local or in-memory backends, and offers no application-level control over key management or per-tenant isolation. The encryption boundary is at the storage service level, not the application level, meaning that any process with storage credentials can read all data in plaintext.

Application-layer encryption with tight coupling. Applications that implement their own encryption typically embed encryption logic directly into data access paths, creating tight coupling between the encryption code and the storage logic. Adding encryption to a new backend requires modifying the new backend's implementation. Removing or changing the encryption scheme requires modifying every backend. This violates the open-closed principle and makes the system brittle.

All-or-nothing migration. When adding encryption to an existing system with plaintext data, most approaches require a batch migration: stop writes, encrypt all existing data, then restart with encryption enabled. This causes downtime proportional to the data volume. For multi-tenant systems with terabytes of data, this downtime may be unacceptable.

Inadequate tenant isolation. Systems that encrypt data at rest often use a single encryption key for all tenants. If the key is compromised, all tenants' data is exposed. Even systems that use per-tenant keys often derive them in an ad-hoc manner without a principled key hierarchy that supports rotation, auditing, and hardware-backed key protection.

Format incompatibility. Binary ciphertext output from standard encryption algorithms is not valid UTF-8 and cannot be stored directly in string-typed storage columns (e.g., DynamoDB String attributes). Naive base64 encoding of ciphertext loses the ability to distinguish encrypted values from plaintext, complicating migration and debugging.

2.2 Limitations of Prior Art

Transparent Data Encryption (TDE) as implemented in relational databases (Oracle TDE, SQL Server TDE) operates at the storage engine level and is specific to a single database product. TDE cannot be applied as a decorator over arbitrary key-value backends, does not support dual-read migration, and does not provide per-tenant key derivation for multi-tenant workloads.

Application-level field encryption libraries (e.g., AWS Encryption SDK, Google Tink) provide cryptographic primitives but do not implement the decorator pattern over storage interfaces. The application developer must manually call encrypt/decrypt at each storage access point, and there is no built-in mechanism for dual-read migration or self-describing ciphertext formats.

Proxy-based encryption (e.g., CipherCloud, Voltage SecureData) interposes a proxy between the application and the storage backend. These systems introduce network latency for every operation, require separate deployment and management of the proxy infrastructure, and do not compose with the application's existing storage abstraction layer.

Envelope encryption as described in cloud KMS documentation (AWS KMS, Google Cloud KMS) defines a key hierarchy where a master key encrypts data keys, but does not specify how to implement this as a transparent decorator over arbitrary storage backends, how to handle migration from plaintext, or how to derive per-tenant keys from a single master hierarchy.

None of these approaches provide the combination of: (1) a transparent decorator that wraps any backend implementing a common storage trait; (2) a self-describing ciphertext format compatible

with string-typed backends; (3) a dual-read migration mode enabling zero-downtime transition; (4) per-tenant key derivation via HKDF with tenant-hash context; and (5) composability with other decorators in a stacked architecture.

3 Summary

The present invention provides a system and method for transparent encryption of data at rest through a decorator layer that wraps any key-value storage backend, with dual-read migration for zero-downtime transition and per-tenant key isolation through a hierarchical key derivation scheme. The invention comprises:

1. **Transparent encrypting decorator.** An `EncryptingNamespacedStore` decorator that implements the same `NamespacedStore` trait (or equivalent key-value storage interface) as the underlying backend. The decorator wraps any concrete backend—Sled, DynamoDB, InMemory, or any future implementation—without requiring modifications to the backend. All read and write operations pass through the decorator transparently: writes encrypt the value before delegating to the inner backend; reads decrypt the value after retrieving it from the inner backend. The application code and the backend code are both unaware of the encryption layer.

2. **Self-describing ciphertext format.** Encrypted values are stored in the format `ENC:<base64(nonce||ciphe` where `ENC:` is a fixed ASCII prefix, and the payload is standard base64 encoding of the concatenation of the random nonce, the AES-256-GCM ciphertext, and the authentication tag. This format is valid UTF-8, can be stored in any string-typed storage column, and is self-identifying: the presence of the `ENC:` prefix unambiguously distinguishes encrypted values from plaintext.

3. **Dual-read migration mode.** On read, the decorator first checks for the `ENC:` prefix. If present, it decodes and decrypts the value. If absent, it returns the value unchanged as plaintext. This enables a gradual migration strategy: existing plaintext data is readable immediately without conversion; all new writes are encrypted; over time, as data is updated through normal application operations, the proportion of encrypted data increases monotonically toward 100%. No batch migration, no downtime, and no flag day are required.

4. **Per-tenant key derivation.** Each tenant is identified by a tenant hash computed as $\texttt{tenant_hash} = \text{SHA-256}(\texttt{tenant_identifier})$. A per-tenant Data Encryption Key (DEK) is derived from the master Key Encryption Key (KEK) using HKDF (HMAC-based Key Derivation Function) with the tenant hash as the context parameter: $\texttt{DEK}_t = \text{HKDF}(\texttt{KEK}, \texttt{tenant_hash}_t)$. This ensures that each tenant's data is encrypted with a unique key, and compromise of one tenant's DEK does not affect other tenants.

5. **Multi-layer key hierarchy.** The system implements a three-tier key hierarchy: (a) an AWS KMS Customer Master Key (CMK) that never leaves the hardware security module; (b) a Key Encryption Key (KEK) that is generated locally, encrypted (wrapped) by the CMK, and stored in a dedicated `DekStore` DynamoDB table; and (c) per-tenant DEKs derived from the KEK via HKDF. The KEK is decrypted (unwrapped) at application startup by calling KMS, and the plaintext KEK is held only in process memory.

6. **Namespace-aware encryption.** Each logical namespace in the storage system (schemas, metadata, main data, transforms, trust graphs, etc.) can use independent key derivation

paths by incorporating the namespace identifier into the HKDF context alongside the tenant hash. This ensures that compromise of one namespace's key material does not expose data in other namespaces.

7. **Decorator composability.** The encrypting decorator conforms to the same trait interface as the backends it wraps. This means it can be composed with other decorators—caching decorators, logging decorators, metrics decorators—in any order. The composition is transparent: each decorator wraps the interface below it and presents the same interface above it. No decorator needs to know whether it is wrapping a raw backend or another decorator.

8. **Key rotation support.** When the KEK is rotated, a new KEK is generated, encrypted under the CMK, and stored alongside the old KEK in the `DekStore`. New per-tenant DEKs are derived from the new KEK. The old KEK is retained so that existing data encrypted under old DEKs can still be decrypted. As data is re-encrypted through normal write operations or a background re-encryption process, it migrates to keys derived from the new KEK. Once all data has been re-encrypted, the old KEK can be retired.

4 Detailed Description

4.1 System Architecture Overview

The system comprises six components (see FIG. 1):

1. **Storage Backend(s).** One or more concrete implementations of a key-value storage interface (the `NamespacedStore` trait). Each backend provides operations: `get(namespace, key)`, `set(namespace, key, value)`, `delete(namespace, key)`, and `list_keys(namespace)`. Backends include Sled (embedded B-tree), DynamoDB (cloud-hosted), and InMemory (hash map). The backend is unaware of encryption.

2. **Encrypting Decorator.** The `EncryptingNamespacedStore` struct that wraps any backend implementing the `NamespacedStore` trait. It holds a reference to the inner backend and a reference to the key provider. On each operation, it intercepts the call, performs encryption or decryption as appropriate, and delegates to the inner backend.

3. **Key Provider.** A component that supplies per-tenant DEKs to the encrypting decorator. The key provider holds the plaintext KEK in memory and derives DEKs on demand using HKDF with the tenant hash and namespace as context. The key provider abstracts the key hierarchy so that the decorator does not need to know whether keys come from KMS, a local file, or any other source.

4. **KMS Integration.** An AWS KMS client that performs envelope encryption of the KEK. At startup, the system calls `kms:Decrypt` to unwrap the KEK. The plaintext CMK never leaves the KMS hardware security module. The wrapped KEK is stored in the `DekStore` table.

5. **DekStore.** A DynamoDB table that stores wrapped KEKs indexed by key identifier and version. Each entry contains the encrypted KEK blob, the KMS key ARN used to encrypt it, a version number, and a creation timestamp. Multiple versions may coexist during key rotation.

   The DekStore is a key-value table with the following schema: partition key is the tenant hash (SHA-256 of tenant identifier), sort key is a KEK version identifier (monotonically increasing

integer). Each record stores: the Data Encryption Key (DEK) encrypted (wrapped) by the corresponding KEK version, the AWS KMS key ARN used for wrapping, a creation timestamp, and an optional expiration timestamp. This schema enables efficient per-tenant key lookup and supports concurrent KEK versions during rotation.

6. **Tenant Isolation Layer.** The system that computes tenant hashes from tenant identifiers and ensures that all storage operations are scoped to the correct tenant. DynamoDB partition keys include the tenant hash. S3 object paths include the tenant hash. Lambda function instances are scoped to a single tenant per invocation via the `X-User-Hash` header.

4.2 The Transparent Decorator Pattern

The encrypting decorator implements the same storage trait as the backend it wraps. In a Rust implementation, this is expressed as:

```
struct EncryptingNamespacedStore<S: NamespacedStore> {
    inner: S,
    key_provider: Arc<KeyProvider>,
}

impl<S: NamespacedStore> NamespacedStore for EncryptingNamespacedStore<S> {
    fn get(&self, ns: &str, key: &str) -> Result<Option<String>> { ... }
    fn set(&self, ns: &str, key: &str, value: &str) -> Result<()> { ... }
    fn delete(&self, ns: &str, key: &str) -> Result<()> { ... }
    fn list_keys(&self, ns: &str) -> Result<Vec<String>> { ... }
}
```

The decorator is generic over the inner store type `S`. Any type that implements `NamespacedStore` can be wrapped, including other decorators. This is the structural foundation of composability.

Write path. When `set(ns, key, value)` is called on the decorator:

1. The decorator obtains the DEK for the current tenant and namespace from the key provider: $\texttt{DEK} = \text{HKDF}(\texttt{KEK}, \texttt{tenant_hash} \| \texttt{ns})$.
2. A fresh 96-bit random nonce is generated.
3. The value is encrypted using AES-256-GCM with the DEK and nonce, producing ciphertext and a 128-bit authentication tag.
4. The nonce, ciphertext, and tag are concatenated and base64-encoded.
5. The result is prefixed with `ENC:` to produce the stored value.
6. The decorator calls `inner.set(ns, key, stored_value)`.

Read path. When `get(ns, key)` is called on the decorator:

1. The decorator calls `inner.get(ns, key)` to retrieve the stored value.
2. If the value is `None`, it returns `None`.
3. If the value starts with the `ENC:` prefix:

   (a) The base64 payload is decoded.
   (b) The nonce, ciphertext, and tag are extracted.
   (c) The DEK is obtained from the key provider.
   (d) The ciphertext is decrypted and authenticated using AES-256-GCM.
   (e) The plaintext value is returned.

4. If the value does not start with `ENC:` (dual-read migration mode):
   (a) The value is returned unchanged as plaintext.
   (b) No error is raised. The system treats the value as pre-encryption legacy data.

Delete and list operations. The `delete` operation delegates directly to the inner backend, as there is no value to encrypt or decrypt. The `list_keys` operation also delegates directly, as keys are not encrypted (only values are encrypted in the present embodiment).

4.3 Self-Describing Ciphertext Format

The stored format for encrypted values is:

```
ENC:<base64(nonce || ciphertext || authentication_tag)>
```

The `ENC:` prefix serves three purposes:

Identification. It enables the dual-read migration mode by distinguishing encrypted values from plaintext. No plaintext value in the application domain begins with `ENC:`, as this prefix is reserved by convention.

UTF-8 compatibility. Base64 encoding ensures that the entire stored value—prefix plus payload—consists only of ASCII characters and is therefore valid UTF-8. This is critical for storage backends that require string-typed values, such as DynamoDB String attributes. Binary ciphertext cannot be stored in DynamoDB String attributes without encoding; the `ENC:` format solves this transparently.

Debuggability. When inspecting raw storage contents (e.g., scanning a DynamoDB table), the `ENC:` prefix immediately indicates that a value is encrypted, aiding operational debugging without compromising security.

The nonce is 96 bits (12 bytes) as specified by the AES-256-GCM standard. The authentication tag is 128 bits (16 bytes). The ciphertext length equals the plaintext length. The total overhead per encrypted value is the 4-byte `ENC:` prefix plus the base64 expansion factor of $\lceil (12 + n + 16) \times 4/3 \rceil$ characters, where n is the plaintext byte length.

4.4 Dual-Read Migration Mode

The dual-read migration mode is the mechanism by which the system transitions from plaintext storage to encrypted storage without downtime or batch conversion. The invariants of dual-read mode are:

Invariant 1: All writes are encrypted. Once the encrypting decorator is deployed, every `set` operation produces an `ENC:`-prefixed value. There is no code path that writes plaintext through the decorator.

Invariant 2: Reads handle both formats. Every `get` operation checks for the `ENC:` prefix and branches accordingly. Encrypted values are decrypted. Plaintext values are returned unchanged.

Invariant 3: Monotonic progress. The proportion of encrypted values in the store is monotonically non-decreasing over time. Each write converts one value from plaintext (or old ciphertext) to new ciphertext. No operation converts an encrypted value back to plaintext.

Migration lifecycle:

1. **Phase 0: Pre-encryption.** All values are plaintext. The system operates without the encrypting decorator.
2. **Phase 1: Decorator deployment.** The encrypting decorator is inserted into the storage stack. Reads work immediately (plaintext pass-through). New writes are encrypted. No downtime.
3. **Phase 2: Gradual migration.** Over time, as values are updated through normal application operations, they are re-written in encrypted form. A background re-encryption job may optionally accelerate this process by reading and re-writing all keys.
4. **Phase 3: Fully encrypted.** All values carry the `ENC:` prefix. The plaintext pass-through code path is never exercised but remains in place for safety.

The dual-read approach is superior to alternatives because it requires zero downtime, introduces no batch processing jobs that must complete before the system can operate, and is idempotent: deploying the decorator multiple times has no adverse effect.

4.5 Per-Tenant Key Derivation

Each tenant in the multi-tenant system is identified by a tenant identifier (e.g., a user ID, email, or API key). The system computes a tenant hash:

$$\texttt{tenant_hash} = \text{SHA-256}(\texttt{tenant_identifier})$$

The tenant hash is computed as SHA-256(UTF-8(tenant_identifier)) where tenant_identifier is the tenant's canonical email address or unique identifier string, encoded as UTF-8 bytes before hashing.

The tenant hash serves as both a storage partition key and a key derivation context. A per-tenant DEK is derived using HKDF-SHA-256:

$$\texttt{DEK}_t = \text{HKDF-Expand}(\text{HKDF-Extract}(\texttt{salt}, \texttt{KEK}), \texttt{tenant_hash}_t \| \texttt{namespace}, 32)$$

The HKDF derivation uses an empty salt (zero-length byte string) for the extract phase, with the KEK as input keying material and the concatenation of tenant_hash and namespace_identifier (in that order, with a single null byte separator) as the info parameter: DEK = HKDF-Expand(HKDF-Extract("", KEK

The output length is 32 bytes (256 bits) for AES-256. The namespace identifier is concatenated with the tenant hash to produce namespace-specific keys, ensuring that the DEK used for the "schemas" namespace differs from the DEK used for the "main_data" namespace, even for the same tenant.

Properties of this derivation:

- **Tenant isolation.** Different tenants produce different DEKs with overwhelming probability (collision resistance of SHA-256 and HKDF).
- **Namespace isolation.** Different namespaces produce different DEKs for the same tenant.

- **Determinism.** The same tenant and namespace always produce the same DEK from the same KEK, enabling stateless DEK computation without storing per-tenant keys.
- **Non-invertibility.** Given a DEK, it is computationally infeasible to recover the KEK or the tenant identifier.

4.6 Multi-Layer Key Hierarchy

The key hierarchy comprises three tiers (see FIG. 2):

Tier 1: Customer Master Key (CMK). An AWS KMS symmetric key stored in FIPS 140-2 validated hardware security modules. The CMK never leaves the HSM in plaintext. It is used exclusively to encrypt and decrypt the KEK. Access to the CMK is controlled by IAM policies and KMS key policies.

Tier 2: Key Encryption Key (KEK). A 256-bit symmetric key generated locally using a cryptographically secure random number generator. The KEK is immediately encrypted (wrapped) by the CMK via the KMS `Encrypt` API and stored as a ciphertext blob in the `DekStore` DynamoDB table. At application startup, the KEK is decrypted (unwrapped) via the KMS `Decrypt` API and held in process memory. The plaintext KEK is never written to persistent storage.

Tier 3: Data Encryption Keys (DEKs). Per-tenant, per-namespace keys derived from the KEK via HKDF as described above. DEKs are computed on demand and may be cached in process memory with a time-to-live (TTL) to amortize the HKDF computation cost. DEKs are never persisted to storage.

Key rotation procedure:

1. A new KEK (version $v + 1$) is generated and encrypted under the CMK.
2. The new wrapped KEK is stored in the `DekStore` with version $v + 1$.
3. The application is configured to use the new KEK for all new writes.
4. The old KEK (version v) remains available for reads. On read, if decryption with the current KEK fails, the system attempts decryption with previous KEK versions.
5. A background re-encryption job reads and re-writes all values, causing them to be encrypted under DEKs derived from the new KEK.
6. Once re-encryption is complete and verified, the old KEK may be retired (deleted from the `DekStore`).

This procedure enables key rotation without downtime. The dual-read mechanism naturally accommodates values encrypted under different KEK versions, as the `ENC:` format is version-agnostic and the system attempts decryption with all available KEKs.

During key rotation, the system attempts decryption with the current (highest-version) KEK first. If decryption fails due to authentication error, the system iterates through previous KEK versions in reverse chronological order. This fallback sequence terminates either when decryption succeeds or all available KEK versions have been exhausted, in which case the record is flagged as corrupted. The expected case during rotation is that the first or second attempt succeeds, as most records are re-encrypted within one rotation cycle.

4.7 Namespace-Aware Encryption

The storage system organizes data into logical namespaces. In a typical deployment, the namespaces include:

- `schemas` – schema definitions and metadata
- `fields` – individual field values
- `transforms` – registered transform definitions
- `trust` – trust distance graphs
- `atoms` – atomic data units
- `permissions` – access control policies

Each namespace may contain data with different sensitivity levels and different regulatory requirements. By incorporating the namespace into the HKDF derivation context, the system ensures that:

- A compromised DEK for the `fields` namespace cannot decrypt data in the `schemas` namespace.
- Different namespaces can be rotated independently if required by compliance policies.
- Forensic analysis of a breach can be scoped to the specific namespace whose key was compromised.

4.8 Decorator Composability

The encrypting decorator is one instance of a general decorator pattern over the `NamespacedStore` trait. Other decorators include:

- **Caching decorator.** Wraps a backend and caches frequently accessed values in memory, reducing latency and backend load.
- **Logging decorator.** Records all storage operations for debugging and auditing.
- **Metrics decorator.** Emits operation counts, latencies, and error rates to a monitoring system.
- **Retry decorator.** Automatically retries failed operations with exponential backoff for transient errors.

Decorators can be composed in any order. For example:

```
let store = DynamoDbStore::new(config);
let store = EncryptingNamespacedStore::new(store, key_provider);
let store = CachingNamespacedStore::new(store, cache_config);
let store = LoggingNamespacedStore::new(store, logger);
```

In this composition, the logging decorator wraps the caching decorator, which wraps the encrypting decorator, which wraps the DynamoDB backend. Each decorator sees only the `NamespacedStore` trait and is unaware of the layers above or below it. The encrypting decorator operates on the raw backend values (below the cache), ensuring that cached values are already decrypted and that the cache never stores ciphertext.

The order of composition matters and is chosen deliberately:

- Encryption below caching ensures the cache holds plaintext, avoiding redundant decryption on cache hits.
- Logging above caching captures the logical operations (including cache hits), not the physical backend calls.
- The backend is always at the bottom of the stack.

4.9 Per-Tenant Isolation at Every Storage Layer

The encryption decorator is one component of a comprehensive per-tenant isolation strategy that operates at every layer of the storage system:

DynamoDB partition keys. Every DynamoDB table uses a composite primary key that includes the tenant hash as the partition key or a prefix of the partition key. This ensures that DynamoDB's internal partitioning isolates tenant data physically, and that a query for one tenant's data cannot accidentally return another tenant's data.

S3 object paths. When large values are stored in S3 (e.g., binary blobs, large documents), the S3 key includes the tenant hash as a path prefix: `/data/<tenant_hash>/...`. S3 bucket policies can further restrict access by path prefix.

Lambda scoping. Each Lambda invocation is scoped to a single tenant via the `X-User-Hash` header. The handler extracts the tenant hash from the header and passes it through the entire storage stack, including the encrypting decorator, ensuring that the correct per-tenant DEK is used.

Encryption keys. As described above, each tenant has a unique DEK derived from the KEK and tenant hash. Even if two tenants store identical values, the ciphertext will differ because the DEKs differ and each encryption uses a fresh random nonce.

This defense-in-depth approach ensures that tenant isolation does not depend on any single mechanism. Compromise of one layer (e.g., a DynamoDB access control misconfiguration) does not expose data if the encryption layer is intact.

4.10 Authenticated Encryption and Integrity

The system uses AES-256-GCM (Galois/Counter Mode), which provides both confidentiality and integrity. The authentication tag ensures that:

- **Tampering detection.** Any modification to the ciphertext, nonce, or tag will cause decryption to fail with an authentication error. The system treats authentication failure as a fatal error, not as a signal to fall back to plaintext (which would create a downgrade attack vector).
- **Ciphertext integrity.** The authentication tag binds the ciphertext to the nonce and the DEK, preventing ciphertext substitution attacks where an attacker replaces one tenant's ciphertext with another's.

- **Nonce uniqueness.** Each encryption operation uses a fresh 96-bit random nonce. With AES-256-GCM, the probability of nonce collision after 2^{32} encryptions under the same key is approximately 2^{-33}, which is negligible for the expected data volumes. For tenants with extremely high write volumes, the namespace partitioning further reduces the number of encryptions under any single DEK.

It is critical to distinguish between an authentication failure (which indicates tampering or key mismatch and must be treated as an error) and the absence of the `ENC:` prefix (which indicates pre-encryption plaintext and triggers the dual-read pass-through). The system checks the prefix *before* attempting decryption, ensuring that the dual-read mechanism cannot be exploited as a downgrade attack.

5 Figures

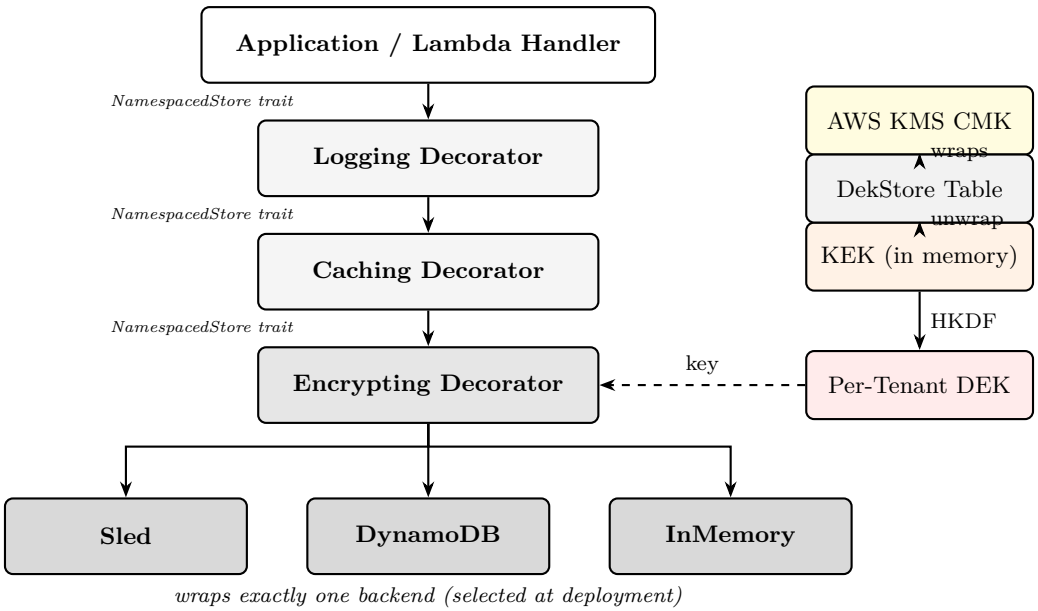


Figure 1: **System Architecture and Decorator Stack.** The application interacts with the `NamespacedStore` trait at the top of a composable decorator stack. The encrypting decorator wraps any backend (Sled, DynamoDB, InMemory). Per-tenant DEKs are derived from the KEK via HKDF; the KEK is stored encrypted by an AWS KMS CMK in the DekStore table.

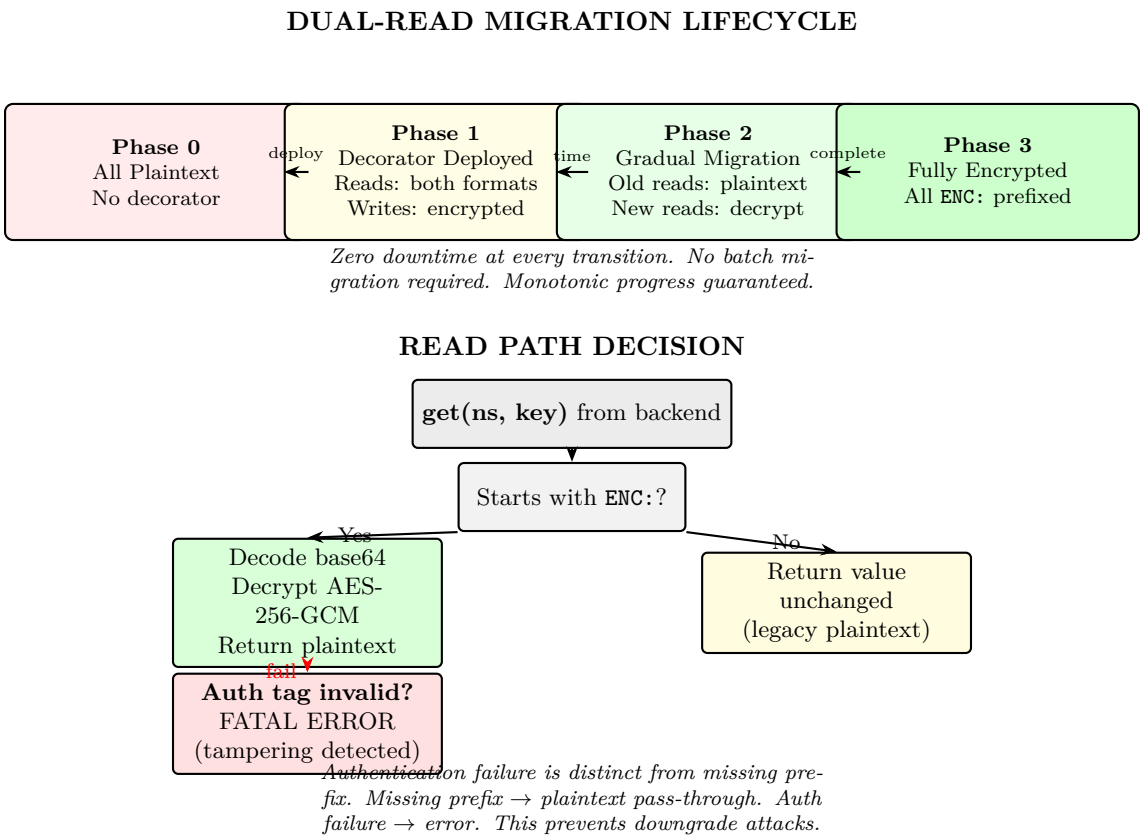
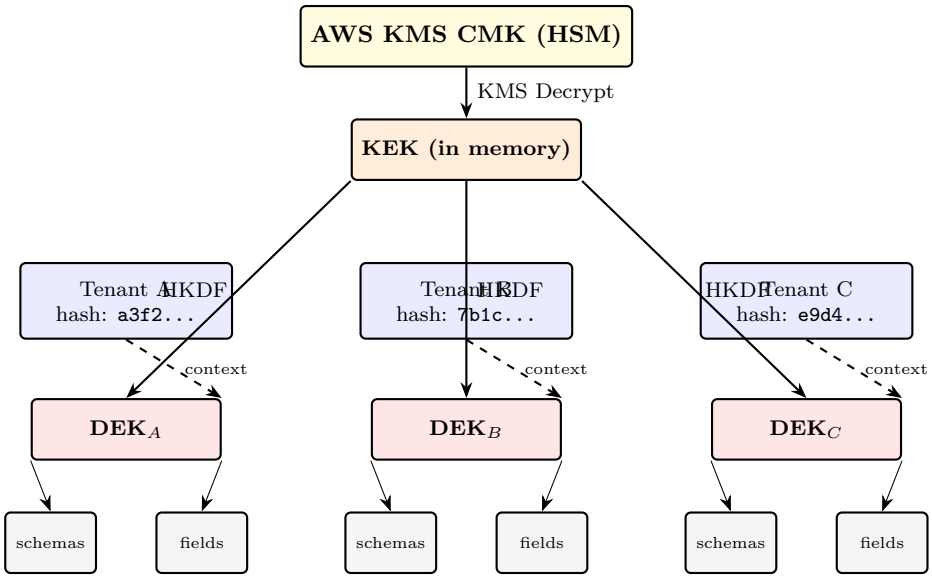


Figure 2: **Dual-Read Migration Lifecycle and Read Path Decision.** Top: the four phases of migration from plaintext to fully encrypted storage, each requiring zero downtime. Bottom: the read path decision tree showing how the `ENC:` prefix distinguishes encrypted values from legacy plaintext, and how authentication failure is treated as a fatal error distinct from the plaintext pass-through path.



Each tenant×namespace combination produces a unique DEK via HKDF(KEK, tenant_hash ‖ namespace). Compromise of one DEK exposes only one tenant's data in one namespace.

Figure 3: **Per-Tenant Key Derivation Hierarchy.** The three-tier key hierarchy: AWS KMS CMK (hardware-backed) protects the KEK (held in memory), which is used with HKDF to derive unique DEKs for each tenant-namespace combination. Each DEK is cryptographically independent.